Empezar

AlgoProject — manual de uso. Actualizado el 2026-10-16 con `tools/manual.py`.

1. 0. Empezar
2. 20. Dónde está todo — el mapa de los datos
3. 21. Vocabulario — qué sabe expresar una instalación de SQX
4. 46. Knowhow en fichas — un hecho por archivo, para gastar pocos tokens al consultarlo
5. 23. Skills — qué hay instalado y qué sobra
6. 10. GitHub — tener el proyecto subido y poder clonarlo
7. 114. Revisión estructural semanal — dónde se está acumulando deuda en el código

El manual entero:

- **01-empezar.pdf** — Empezar
- **02-la-ventana.pdf** — La ventana de escritorio
- **03-datos-costes-y-registro.pdf** — Datos, costes y registro de la búsqueda
- **04-sqx-plantillas-y-proyectos.pdf** — SQX: bloques, plantillas y proyectos
- **05-cribado-oos.pdf** — Cribado fuera de muestra
- **06-lecturas.pdf** — Lecturas de una estrategia
- **07-otros-mercados-y-timeframes.pdf** — Otros mercados y otros timeframes
- **08-montecarlo.pdf** — Monte Carlo y MC Retest
- **09-optimizacion.pdf** — Parámetros: SPP, variantes, WFC, CSCV y WFM
- **10-cierre.pdf** — El cierre: paso 20, exposición y MetaTrader 5
- **11-cartera.pdf** — Cartera: el pool, el universo, las parejas y la búsqueda de fondeo
- **12-en-real.pdf** — En real: la cuenta contra SQX

0. Empezar

Este manual explica **cómo usar el proyecto**, no cómo está programado por dentro. Todo lo que aparece aquí son comandos que puedes copiar y pegar en una terminal.

Qué es esto

StrategyQuant X genera y testea estrategias. Este proyecto hace las matemáticas que SQX no hace: saca los datos de SQX, los analiza y te devuelve gráficas y conclusiones para que decidas qué estrategias usar y con qué filtros generarlas.

El trabajo está partido en fases. Cada una tiene su carpeta:

carpeta	de qué va	ejemplo
`sqx/`	hablar con SQX: sacar datos, inspeccionar proyectos, repararlos	exportar las métricas de un databank
`tasks/`	analizar **poblaciones enteras**: miles de estrategias a la vez	¿qué métrica del histórico predice el futuro?
`strategies/`	analizar **una** estrategia a fondo	traducir una estrategia a Python
`portfolio/`	carteras	—

Si tu pregunta es sobre miles de estrategias, es `tasks/`. Si es sobre una, es `strategies/`.

La primera vez, en esta máquina

Ya está hecho. Solo lo necesitas si cambias de ordenador:

```
cd ~/Desktop/AlgoProject
cp config/machine.example.yaml config/machine.yaml   # y editar las rutas de dentro
python3 -m pip install -r requirements.txt
```

`config/machine.yaml` es el **único** archivo que cambia entre ordenadores: dice dónde está SQX, dónde van los datos y dónde está el navegador. Ningún otro archivo tiene rutas dentro.

Cómo se ejecuta cualquier cosa

Siempre igual, y siempre desde la raíz del proyecto, como módulo (`-m`), nunca por ruta:

```
cd ~/Desktop/AlgoProject
python3 -m paquete.modulo --unos --flags
```

Todos los comandos aceptan `--help`, que te lista los flags sin ejecutar nada:

```
python3 -m studies.screening.isOos.report --help
```

Dónde acaban las cosas

Nada se guarda dentro del proyecto. Todo va a `~/Desktop/AlgoData`, que tiene tres zonas y **cada una se comporta distinto**. Esto importa:

zona	qué pasa cuando vuelves a exportar
`metrics/<proyecto>/<databank>/`	**se borra lo anterior y se escribe lo nuevo.** Hay un solo CSV vigente por databank, para que nunca tengas dudas de cuál es el bueno
`raw/<proyecto>/<databank>/<fecha>/`	**no se toca nada.** Cada exportación de trades y barras es de un día y se queda ahí para siempre
`reports/<proyecto>/<databank>/<fecha>/`	**se acumulan.** Los análisis nunca se borran

La lógica: un CSV lo puedes volver a sacar de SQX cuando quieras. Las conclusiones que sacaste un día no. Por eso los reportes se guardan todos y los CSV no.

`~/Desktop/AlgoData/INDEX.md` es la lista de todo lo que ya existe. **Míralo antes de exportar nada**, porque una exportación cuesta minutos de SQX y puede que ya la tengas.

Las reglas que no puedes romper

Estas son de SQX, no del código, y romperlas destruye trabajo de verdad:

1. **SQX borra del disco las estrategias que no tiene en memoria.** Cada vez que sincroniza. Si vas a hacer algo que reinicie SQX, copia antes `user/projects` a otro sitio.
2. **Nunca lances `sqcli` en el SQX principal con la ventana abierta.** No responde. Para eso está el worker, que los scripts arrancan y paran solos.
3. **Nunca mates procesos con `pkill -f StrategyQuantX`** — ese patrón también coge tu propia terminal. Se mata por PID o no se mata.
4. **Nunca empieces un build.** La ventana de SQX es tuya y de nadie más; los scripts no lanzan builds y no deben hacerlo.
5. **Nunca edites un `project.cfx` que SQX tenga abierto.** SQX reescribe el archivo al guardar y al salir, y tu cambio desaparece sin avisar.

Cuándo puedes tener SQX abierto

lo que haces	¿puedes tener la GUI de SQX abierta?
analizar datos ya exportados	**sí**, no toca SQX para nada
exportar métricas, trades o barras	**sí**: los scripts usan el worker, que es una copia aparte
inspeccionar proyectos (leer `project.cfx`)	**sí**, es solo lectura
reparar un proyecto	**no**, hay que cerrar SQX antes

Si algo falla

- **`FileNotFoundError` con una ruta de `AlgoData`** — no has exportado todavía. Mira el paso "Antes de empezar" de la página del módulo.
- **Un CSV que sale vacío o solo con la cabecera** — el worker exportó antes de terminar de cargar. Vuelve a lanzarlo; el script espera, pero si SQX estaba a medias puede pasar.
- **El comando se queda colgado varios minutos** — normal si toca SQX. Arrancar el worker son ~30 s y exportar 10.000 estrategias son minutos. Si pasa de 15 minutos, algo va mal.
- **`bin/sqx-worker.sh … stop` tarda más de 5 minutos** — normal: si SQX no ha salido a los 5 minutos, la orden espera mientras su log se mueve (al salir guarda todos los databanks, y con varios proyectos son minutos), hasta 15. Si lleva 60 s en silencio y sigue ahí, está colgado y la orden **lo mata por PID** (dueño, 2026-10-06): ya

no hace falta matarlo desde el monitor del sistema. Solo si sobrevive incluso a eso acaba con «⚠ … STILL RUNNING even after the kill» y código de salida 1: lo que la llamó se para con error y no lee los databanks a medio escribir.

- **«REFUSING: another start or run of … is in progress»** — otra sesión está arrancando ese mismo worker en este momento. No es un fallo: espera a que acabe y mira quién lo tiene (`bin/sqx-worker.sh --role <rol> check`) antes de volver a lanzar.
- **«worker failed to start»** — SQX no abrió su puerto en 2 minutos (lo normal son 1-7 s). El script mata el proceso que lanzó y no deja candado; mira el final de `user/log/worker-daemon.log` del worker y vuelve a arrancar.

Índice

- 1. Análisis IS/OOS — qué métrica del histórico predice el futuro

20. Dónde está todo — el mapa de los datos

Qué pregunta responde

¿Dónde ha quedado esto que acabo de generar? Y la de al lado, que es la que de verdad hace perder tiempo: *¿esto ya existe, o lo vuelvo a exportar?*

Hay **dos sitios**, y la frontera entre ellos es la regla dura 7 del proyecto:

dónde	qué vive ahí	cuánto pesa
`~/Desktop/SQX*/user/projects/`	**las estrategias vivas**, dentro de SQX. Las que ves en la GUI	4,2 GB en el maestro
`~/Desktop/AlgoData/`	**todo lo que este proyecto produce**: exports, backtests, métricas, informes	7,3 GB
`~/Desktop/AlgoProject/`	**solo código y texto**. Nunca datos	30 MB

⚠ **En el repositorio no entra ni un dato.** `.gitignore` bloquea `*.csv`, `*.parquet` y `*.sqx`, y `checks.py` falla si algún módulo escribe fuera de `core/paths.py`. Si buscas un número, no está en el repo: está en `AlgoData`.

Cuándo lo usas, y cuándo no

Lo usas cuando no encuentras algo, cuando vas a exportar y quieres comprobar si ya está, o cuando te preguntas qué se puede borrar. **No lo usas** para saber qué contiene cada fichero: eso lo dice `~/Desktop/AlgoData/INDEX.md`, que lleva una línea por export con sus filas y sus columnas.

Antes de empezar

Nada. Es lectura.

Cómo se ejecuta

```
cat ~/Desktop/AlgoData/INDEX.md          # qué hay, export por export
python3 -m perf.disk.report --quick      # qué pesa cada rama y contra qué presupuesto
```

Qué produce

Nada. Este capítulo describe lo que producen los demás.

Las estrategias — hay cuatro sitios y conviene no confundirlos

qué	dónde	quién lo escribe
Las vivas, las del maestro	`~/Desktop/SQX/user/projects/<proyecto>/databanks/<databank>/*.sqx`	SQX. 7.546 hoy
Las fabricadas (las 5.000 permutaciones)	`AlgoData/pipeline/<proyecto>/<estrategia>/sqx/P*.sqx`	`sqx.variants.make`
Las copiadas a un export	`AlgoData/raw/<proyecto>/<databank>/<fecha>/strategies/*.sqx`	`sqx.export.*`, al exportar trades
El snapshot de seguridad	`AlgoData/snapshots/<fecha>/master/projects/`	copia manual antes de algo destructivo

⚠ **Las del maestro son volátiles y las demás no.** Cada sync de SQX borra del disco los `.sqx` que no tiene en memoria, y hay auto-sync horario: por eso existe la rama `snapshots/`, y por eso el donante congelado vive en `AlgoData/projectsBackup/`, no en el install. Regla dura 1.

⚠ **Las fabricadas se borran a propósito.** Son 14 KB cada una y 5.000 por estrategia madre; el paso de limpieza las tira cuando ya se midieron. **Lo que sobrevive es el registro**, que pesa kilobytes y dice qué había. Eso es lo que hace que el borrado sea auditable en vez de una pérdida.

Los backtests y las métricas

qué	dónde	vida
Las métricas de un databank (una fila por estrategia, IS y OOS emparejados)	`AlgoData/metrics/<proyecto>/<databank>/metrics.csv`	**una sola copia**, se reemplaza al refrescar
Las operaciones, todas (12 columnas tipadas)	`AlgoData/raw/<proyecto>/<databank>/<fecha>/trades.parquet`	fechada, inmutable
El perfil SPP (la tabla de permutaciones)	`AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/`	fechada, inmutable
El panel de un lote de variantes	`AlgoData/pipeline/<proyecto>/<estrategia>/retest.csv`	se rehace al reejecutar
Ese panel unido al diseño (contrato C3)	`AlgoData/pipeline/<proyecto>/<estrategia>/metrics.parquet`	**esta es la tabla del estudio**
Las barras	`AlgoData/bars/<feed>/M1.parquet`	la única copia. M30/H1/H4/D1 se resamplean
Las barras resampleadas (caché)	`AlgoData/barsDerived/<feed>/<TF>-<huella>.parquet`	se puede borrar, se regenera

`metrics/` guarda una sola copia a propósito, para que "cuál es el CSV bueno" no pueda ser una pregunta. `raw/` va fechado porque un export de hoy y uno de hace un mes son datos distintos, no versiones del mismo.

Los informes y los veredictos

qué	dónde
Informes de un databank	`AlgoData/reports/<proyecto>/<databank>/<fecha>/<estudio>/` — una subcarpeta por estudio (`gate/` , `isOos/` , `monteCarlo/` , `mcRetest/` , `exposure/` ...) con `<estudio>.html/.md/.json` , `estrategias/` y `verdict.csv` . Qué nombre tenía antes cada una: `docs/MAPA-DE-CARPETAS.md`
El brief de diseño de una madre (contrato C1)	`AlgoData/reports/<proyecto>/<databank>/<fecha>/spp/design_brief_<estrategia>.json`
El gráfico del walk forward correlation	`AlgoData/pipeline/<proyecto>/<estrategia>/estudios/wfc.html`
Su veredicto en números	`AlgoData/pipeline/<proyecto>/<estrategia>/wfc.json`
El libro mayor de una madre	`AlgoData/pipeline/<proyecto>/<estrategia>/state.json`

`reports/` **no se borra nunca**. El CSV se puede regenerar desde SQX; el razonamiento no.

Una carpeta de pipeline por dentro

Es donde vive un estudio completo de una estrategia madre:

```
AlgoData/pipeline/XAUUSD/Strategy_17-9-39/
├── plan.csv            el diseño entero: qué combinaciones y de qué estrato
├── design.json         el resumen del diseño: presupuesto por estrato y faltante
├── sqx/P*.sqx          las 5.000 fabricadas.  ← LO ÚNICO PESADO. Se borra
├── manifest.parquet    contrato C2: qué es cada fichero, leído DEL DISCO
├── build.json          cuántos ficheros y cuántos bytes se escribieron
├── retest.csv          el panel tal y como lo escupe SQX
├── ran.json            cuántas se cargaron y cuántas volvieron
├── metrics.parquet     contrato C3: el panel unido al diseño.  ← LA TABLA
├── collected.json      los controles, y si alguno no volvió
├── estudios/wfc.html   EL GRÁFICO (y cscv.html, cloud.html: los estudios del lote)
├── wfc.json            rho, su intervalo, el veredicto
└── state.json          el libro mayor. Sobrevive al borrado de sqx/
```

Todo menos `sqx/` **pesa unos 500 KB.** `sqx/` pesa 70 MB con 5.000 variantes. Por eso el borrado solo toca esa carpeta, y solo después de que `metrics.parquet` esté escrito y verificado.

El resto de ramas

rama	qué es	¿se puede borrar?
`projectsBackup/`	copias congeladas: `project.cfx` donantes, las madres, y las configs de los installs. El punto fijo del que salen los proyectos nuevos	**no**
`snapshots/`	`user/projects` copiado antes de algo destructivo, **sin las carpetas `log/` de los proyectos** (`rsync -a --exclude='log/'`). Vive hasta que se comprueba que el reinicio no perdió nada, y entonces **se borra** (dueño, 23-09-2026): el del 21-09 eran 3,5 GB sin un solo fichero que el maestro vivo no tuviera	sí, en cuanto se verifica
`logs/`	los logs de SQX comprimidos, de los tres installs. Un log gigante se guarda *condensado* (`*.condensed.log.gz`): sin trazas Java ni las líneas por estrategia del EdgeDecay, que eran el 97 %	**no** — es lo que hace seguro podar los vivos
`cache/`	cachés de los paneles retirados (Monte Carlo, cross-market)	sí, ya no los escribe nadie
`profiling/`	el catálogo de lo que cuesta cada cosa y cuánto ocupa el disco	**no**
`strategyPermutations/`	las variantes fabricadas a mano de una estrategia (fuera del pipeline)	cuando el pipeline las haya cubierto

Cómo se sabe qué produjo un fichero

Cada directorio lleva un `manifest.json` con el comando que lo escribió, cuándo y con qué entradas. Uno sin manifiesto no es reproducible, y el auditor lo marca como hallazgo.

```
cat ~/Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/manifest.json
```

Presupuestos — qué pasa cuando una rama crece

`perf/config.yaml` le pone techo a cada rama, y el pipeline **se niega a empezar otra madre** si el total o alguna rama está por encima. No es un aviso: para la corrida.

```
AlgoData: 7.27 GB de 90 GB (8%) en 70 ramas
           ok      4.78 GB /   20 GB     24%  snapshots
           ok      1.96 GB /   40 GB      5%  raw
           ok      0.00 GB /   15 GB      0%  variants
```

Una rama nueva sin presupuesto sale como `unbudgeted` en vez de pasar en silencio, a propósito: que aparezca algo que crece tiene que verse.

Qué NO te dice

- **No te dice qué hay dentro de cada export.** Eso es `AlgoData/INDEX.md` , una línea por export con sus filas y columnas.
- **No te dice qué se puede borrar con seguridad.** Eso lo calcula `python3 -m perf.disk.report` , que propone y nunca borra.
- **No cubre el árbol de SQX.** Lo que hay dentro de un `.sqx` o de un `project.cfx` está en `knowhow/sqx-format/` .

Si algo falla

síntoma	qué pasa
no encuentras un export que juras haber hecho	míralo en `AlgoData/INDEX.md`; si no está, no se escribió
un directorio sin `manifest.json`	no es reproducible. Vuelve a exportarlo antes de usarlo
`AlgoData N GB de 90 GB` y se para una corrida	`python3 -m perf.disk.report` dice qué rama y qué sobra
faltan `.sqx` en un databank del maestro	regla dura 1: un sync los borró. Mira `knowhow/databanks/sync-deletes-unloaded-files.md`

21. Vocabulario — qué sabe expresar una instalación de SQX

Qué pregunta responde

Antes de diseñar una estrategia hay que saber si SQX conoce la idea. "Quiero que el cierre cruce el canal de Keltner" solo se puede construir si esta instalación tiene bloques de Keltner, y —esto es lo que casi nadie comprueba— si alguno de sus **grupos aleatorios** los contiene.

Son dos preguntas distintas y la segunda es la que rompe las tardes. **Una plantilla no referencia bloques: referencia grupos.** Un bloque que no está en ningún grupo existe, se ve en el listado, y es inalcanzable desde una plantilla. Esta herramienta contesta las dos de un tirón, y además compara instalaciones: si autoras en el conductor y construyes en el custodio, lo que no esté en los dos no existe.

Cuándo lo usas, y cuándo no

Úsalo **antes** de diseñar una plantilla o de encargar un bloque nuevo, y cuando llegue una plantilla de otro ordenador y quieras saber si aquí se puede construir.

No sirve para saber si una idea es buena, ni si un bloque hace lo que su nombre promete. Solo dice qué se puede expresar. Tampoco mira dentro de las plantillas ya escritas: eso es `sqx/inspect/template_check.py`.

Antes de empezar

Nada. Lee tres ficheros XML del disco, no arranca SQX y no necesita ningún worker levantado. Se puede correr con la GUI del maestro abierta y trabajando, porque no escribe en ninguna instalación.

Cómo se ejecuta

```
python3 -m sqx.inspect.vocabulary                          # resumen del conductor
python3 -m sqx.inspect.vocabulary keltner                  # buscar un indicador
python3 -m sqx.inspect.vocabulary --role master            # mirar otra instalación
python3 -m sqx.inspect.vocabulary --diff custodian         # qué le falta al custodio
python3 -m sqx.inspect.vocabulary --snapshot               # guardar el inventario
```

flag	obligatorio	qué hace
`TERM`	no	subcadena a buscar, sin distinguir mayúsculas. Sin él, imprime el resumen
`--role`	no	`master`, `conductor` o `custodian`. Por defecto el conductor
`--diff ROLE`	no	dice qué le falta a esa instalación **contra** la de `--role`
`--snapshot`	no	además escribe el inventario bajo `AlgoData/templates/vocabulary/`

Tarda menos de un segundo. Son 2,6 MB de XML y nada más.

Qué produce

Por consola, siempre. Con `--snapshot`, además:

```
~/Desktop/AlgoData/templates/vocabulary/<instalacion>-<AAAA-MM-DD>.json    (~180 KB)
```

El fichero lleva la fecha en el nombre y **no se sobreescribe**: el valor está en la comparación —qué le falta al custodio, qué cambió desde la semana pasada— y un único fichero "actual" no contesta ninguna de las dos.

Cómo se lee el resultado

El resumen sin argumentos, tal cual sale hoy:

```
SQX_w1: 849 native + 171 own blocks, 20 groups
  Condition pools (15): BookTriggers, BreakoutLong, CrossMAsAbove, CrossMAsBelow,
    DirectionalMomentumFilters, EntropyFilter, EntropyFilter2, GroupIBS, GroupRSIs, HurstLarger,
    MeanReversionFilters, PeriodsFastMABelow, StructuralBreakFilters, TrendRegimeFilters,
    VolatilityRegimeFilters
  Value pools     (4): BollingerBands_Lower, BreakoutRand, GroupMAs, RandomPrices
  EMPTY, unusable (1): RandConditions
```

Tres cosas que leer ahí:

- **849 nativos + 171 propios.** Los nativos vienen con SQX; los propios los autoraste tú y viven en `customBlocks.xml`.
- **Los pools son lo que una plantilla puede usar.** 15 de condición y 4 de valor. Los de valor son el cuello de botella: todas las formas de entrada por stop necesitan uno, y solo hay cuatro.
- **`EMPTY, unusable.`** `RandConditions` tiene cero items. Una plantilla que apunte un hueco ahí **no da error**: construye y no muestrea nada. Es un fallo mudo, y por eso sale en línea aparte.

Un ejemplo completo

La pregunta real: *"quiero una estrategia en la que el cierre de una barra cruce por encima del canal de Keltner"*. ¿Se puede?

```
$ python3 -m sqx.inspect.vocabulary keltner
KCBarClosesAboveUpper  [native/Conditions/Keltner Channel] -> boolean
    Bar closes above Keltner Channel(@Chart@#Period#, #Deviation#).Upper[#Shift#]
    pooled by: NO GROUP POOLS IT — unusable in a template
KCBarClosesAboveLower  [native/Conditions/Keltner Channel] -> boolean
    Bar closes above Keltner Channel(@Chart@#Period#, #Deviation#).Lower[#Shift#]
    pooled by: NO GROUP POOLS IT — unusable in a template
...
KeltnerChannel  [native/Values/Indicators] -> price
    Keltner Channel(@Chart@#Period#, #Deviation#).#Line#[#Shift#]
    pooled by: BollingerBands_Lower
MTKeltnerChannel  [native/Values/Indicators] -> price
    MT Keltner Channel(@Chart@#Period#, #Deviation#).#Line#[#Shift#]
    pooled by: BollingerBands_Lower
```

Salen **18 bloques**, y cada línea decide algo:

1. Hay una categoría nativa entera, `Conditions/Keltner Channel`, con 16 condiciones ya hechas. `KCBarClosesAboveUpper` es literalmente la idea. **No hay que autorar ningún bloque.**
2. Pero las 16 dicen `NO GROUP POOLS IT`. Existen y son inalcanzables: hace falta crear un grupo de condición que las contenga, con `sqx-random-group`. Ese, y no el bloque, es el trabajo real.
3. El indicador como **valor** sí está pooled, por `BollingerBands_Lower` — un grupo cuyo nombre engaña, porque contiene once bandas distintas, Keltner entre ellas. O sea que como *nivel de precio* el Keltner ya se puede usar hoy; como *condición de entrada*, todavía no.

4. `#Line#` en el display es un hueco tipado del propio bloque: banda superior, media o inferior. La ambigüedad no es del diseñador, la impone SQX, y hay que resolverla antes de generar.

Y la comparación entre instalaciones:

```
$ python3 -m sqx.inspect.vocabulary --diff custodian
SQX_w2 lacks, against SQX_w1:
  custom blocks (0): none
  groups (0): none
```

Las tres instalaciones de esta máquina tienen hoy el mismo vocabulario. Cuando eso deje de ser cierto —y dejará de serlo en cuanto se autore algo en una sola— esta salida es la que evita construir en el custodio una plantilla que solo existe en el conductor.

Qué NO te dice

- **No dice si un bloque hace lo que su nombre dice.** `KCBarClosesAboveUpper` se lee y se cree; nadie ha verificado su cálculo.
- **No dice si una plantilla construirá.** Que los bloques existan y estén pooled es necesario y no suficiente: lo único que lo prueba es una construcción real que saque estrategias.
- **No mira dentro de las plantillas.** Si quieres saber si las estrategias que un proyecto construyó llevan de verdad los bloques que su plantilla fija, eso es `template_check.py`.
- **Un grupo con items no es un grupo bueno.** Cuenta cuántos hay, no si tienen sentido juntos: `BollingerBands_Lower` mezcla once bandas y la herramienta lo da por bueno.

Si algo falla

`KeyError: 'custodian'` — `config/machine.yaml` no define ese rol. Es la respuesta honesta en una máquina con solo dos instalaciones; usa `--role conductor`.

`FileNotFoundError` en `internal/web/SQWIZARD/...` — la ruta de `machine.yaml` no apunta a la carpeta raíz de un SQX, la que contiene `internal/` y `user/`.

46. Knowhow en fichas — un hecho por archivo, para gastar pocos tokens al consultarlo

Qué pregunta responde

Dónde está escrito un hecho que ya costó descubrir (cómo se comporta SQX, un formato, un coste, una lección estadística), y cómo encontrarlo **leyendo lo mínimo**. Desde el 2026-09-25 el `knowhow/` no son nueve archivos largos, sino 135 fichas pequeñas en 12 carpetas. Cada ficha cuenta un solo hecho, con la regla arriba y la evidencia debajo.

Cuándo lo usas, y cuándo no

- Lo usas cada vez que una sesión (tuya o de un agente) necesita un hecho del knowhow, y cada vez que alguien descubre uno nuevo y tiene que escribirlo.
- `tools/knowhowmap.py` se corre después de crear, renombrar o cambiar la pregunta `q:` de una ficha.
- **No sirve** para lo que está roto o pendiente (eso va en `OPEN.md`), ni para cómo se usa un comando (eso es este manual).

Antes de empezar

Nada. No toca SQX ni datos: solo lee y escribe archivos markdown dentro de `knowhow/`.

Cómo se ejecuta

Para **consultar** un hecho, en este orden (cada paso cuesta menos que el siguiente):

```
grep -rh '^q:' knowhow/costs/ | grep -i swap          # 1. qué ficha responde
sed '/^## Evidence/q' knowhow/costs/swap-types.md     # 2. solo la regla
cat knowhow/costs/swap-types.md                       # 3. la evidencia, si hace falta discutirla
```

Si no sabes en qué carpeta está, `grep -ril '<palabra>' knowhow/`. La lista de carpetas y el formato de ficha están en `knowhow/INDEX.md`.

Para **regenerar los índices** de cada carpeta después de tocar fichas:

```
python3 tools/knowhowmap.py
```

No tiene flags. Tarda menos de un segundo y no toca SQX.

Qué produce

Reescribe `knowhow/<carpeta>/INDEX.md` en cada una de las 12 carpetas: una línea por ficha con su nombre y la pregunta que responde. **Sobrescribe** esos índices enteros; no se editan a mano.

Cómo se lee el resultado

La consulta del paso 1 devuelve la línea `q:` de la ficha que responde:

```
q: SQX swap types points percent money formula; percent swap annual or nightly 360; convert points to
percent; triple swap day per feed WEDNESDAY FRIDAY; tripleSwapOn override asset file
```

El paso 2 devuelve la cabecera: la pregunta, la etiqueta (□ medido · □ leído en logs · □ inferido), la fecha y la regla.

```
# `percent` swap is an ANNUAL rate (÷100 ÷360); the triple-swap day is per feed
- One night × nights held (3 on the triple day). `pct_annual = points × tick_size × 36000 /
reference_price` ...
- `tripleSwapOn`: WEDNESDAY for FX, XAUUSD, XAGUSD; FRIDAY for BRENT, DJ30, NIKKEI225, USA500, USATEC
...
```

`knowhowmap.py` imprime cuántas fichas tiene cada carpeta:

```
authoring 9, columns 4, conditions 8, costs 10, databanks 8, eng 11, export 15, locations 4, perf 10,
research 21, sqx-drive 17, sqx-format 18
```

Un ejemplo completo

Lo que cuesta responder diez preguntas reales, antes (leyendo el archivo entero, como decía el índice antiguo) y después (grep de `q:` y cabecera de la ficha), medido el 2026-09-25:

pregunta	antes	después	×
¿`startOnlyTask` funciona headless?	~14.600 tok	~260 tok	57×
¿Cuántos núcleos tiene el servidor?	~27.800 tok	~240 tok	118×
¿Qué columnas trae `orderstocsv`?	~13.100 tok	~240 tok	54×
¿Un sync borra `.sqx`?	~4.000 tok	~440 tok	9×
¿Cómo se mide la memoria de un pool `fork`?	~27.800 tok	~440 tok	64×
¿Dónde viven los costes de una tarea?	~14.600 tok	~350 tok	42×
¿Qué ventana seleccionó una estrategia?	~5.800 tok	~270 tok	22×
¿Cómo se identifica una estrategia?	~13.900 tok	~260 tok	53×
¿Qué tipos de swap hay?	~4.200 tok	~260 tok	16×
¿Por qué `Param Count` sale mal?	~2.900 tok	~230 tok	13×
total	**~128.600 tok**	**~3.000 tok**	**43×**

Qué NO te dice

- Que un hecho siga siendo verdad hoy. La fecha de la ficha dice cuándo se midió, y una ficha □ sigue siendo una suposición.
- Todo lo que se sabe de un tema. Si una ficha tiene `see:`, los hechos vecinos están en las fichas que nombra.

Si algo falla

`python3 tools/checks.py` revisa tres cosas del knowhow:

- **`knowhow cards`**: una ficha sin `q:` o sin `tag:`, con más de 12 líneas antes de `## Evidence`, con un `see:` que no apunta a ninguna ficha, o de más de 6 KB. La solución es recortarla o partirla en dos fichas.
- **`knowhow links`**: algún archivo del repo cita un `knowhow/...` que no existe. Pasa al renombrar o borrar una ficha: corrige la cita.
- **`knowhow indexes`**: un `INDEX.md` de carpeta no coincide con sus fichas. Corre `python3 tools/knowhowmap.py`.

La regla de escritura: **se edita la ficha que existe, nunca se añade al final**. Si un hecho cambia, se reescribe la regla y se actualiza `date:`; git guarda la versión anterior.

23. Skills — qué hay instalado y qué sobra

Qué pregunta responde

Cuántas skills hay, qué hace cada una, cuánto cuesta invocarla y cuándo se tocó por última vez. Sirve para dos cosas distintas: que tú veas de un vistazo cuáles se han vuelto innecesarias, y que una instancia nueva de Claude sepa qué tiene disponible sin ir abriendo carpetas.

Cuándo lo usas, y cuándo no

Cuando añadas o quites una skill, y cuando quieras auditar qué está pesando. No sirve para saber si una skill **funciona**: solo lee lo que declara.

Antes de empezar

Nada. Lee carpetas del disco.

Cómo se ejecuta

```
python3 tools/skillmap.py
```

Sin flags. Reescribe `docs/SKILLS.md` de la marca `<!-- generado -->` hacia abajo y **respeta todo lo que haya encima**, que es donde vive el juicio: las familias, las candidatas a retirar, y cómo se paga una skill. Instantáneo.

Qué produce

```
docs/SKILLS.md
```

Dos tablas, una por sitio: las del proyecto (`.claude/skills/`) y las globales (`~/.claude/skills/`). Ordenadas de más pesada a menos, porque las que hay que cuestionar son las caras.

Cómo se lee el resultado

```
| skill              | ~tokens al invocar | ficheros | último cambio | para qué |
| analysis-crossmarket |            2,734 |        1 |    2026-09-21 | ...     |
| template-run         |            1,189 |        1 |    2026-09-22 | ...     |

12 skills, 13,383 tokens de cuerpo en total, 52 KB en disco.
```

Tres columnas que deciden algo:

- **`~tokens al invocar`** es el cuerpo del `SKILL.md`, que solo se carga cuando la usas. La descripción (~90 tokens) está **siempre** en contexto, la invoques o no. Por eso el número de skills casi da igual y el peso de cada una no.
- **`ficheros`**: más de uno significa que tiene material de referencia que carga solo si hace falta. Eso es bueno — es cuerpo que no se paga por defecto.
- **`último cambio`**: una skill que lleva meses quieta y describe un flujo que ya cambió es una trampa, no documentación.

La señal de que una skill hay que partirla es cargar 6.000 tokens para hacer un 20 % de lo que explica. La de que hay que retirarla es que su descripción compita con otra mejor: el riesgo no es el gasto, es que una petición tuya enrute a la equivocada.

Un ejemplo completo

```
$ python3 tools/skillmap.py
docs/SKILLS.md: 12 de proyecto, 4 de global
```

Y lo que sale a la luz al leerlo: las cuatro globales de sqx-lab suman **18.255 tokens**, más que las doce del proyecto juntas (13.383), y ninguna conoce los defaults de esta casa. Están marcadas como candidatas a retirar en la cabecera de `docs/SKILLS.md`.

Qué NO te dice

- **No dice si una skill funciona**, ni si lo que declara sigue siendo cierto. Lee frontmatter.
- **No dice cuántas veces se ha usado.** No hay telemetría; "último cambio" es lo más parecido y no es lo mismo.
- **No mide lo que una skill carga a demanda.** La columna de tokens es el `SKILL.md`; sus ficheros de referencia pueden multiplicar eso si se leen todos.

Si algo falla

`IndexError` al leer un `SKILL.md` — ese fichero no tiene frontmatter entre `---`. Es un error de la skill, no del mapa.

Los cambios a mano desaparecen — los escribiste **debajo** de la marca `<!-- generado -->`. Todo lo que quieras conservar va encima.

10. GitHub — tener el proyecto subido y poder clonarlo

Qué pregunta responde *(obligatoria)*

«¿Lo que hay en GitHub es lo mismo que tengo en este ordenador?» Y si no lo es, lo sube.

Sirve para dos cosas distintas. Una es **copia de seguridad**: el trabajo de meses no vive solo en un disco. La otra es **poder trabajar en otro ordenador**: clonas el repositorio y tienes el proyecto entero, sin tener que copiar carpetas a mano.

Lo que **no** sube nunca son los datos. Los CSV exportados, los `.sqx` y las bases de datos de SQX se quedan en `~/Desktop/AlgoData` y en las instalaciones. El repositorio son el código y la documentación: 12 MB, se clona en segundos.

Cuándo lo usas, y cuándo no *(obligatoria)*

Cuándo sí:

- Al acabar una sesión de trabajo que ha dejado cosas sin commitear.
- **Antes de clonar el proyecto en otro ordenador.** Lo que no esté subido, no aparece en el clon.
- Cuando no sabes si está subido. Preguntarlo cuesta tres segundos.

Cuándo no:

- Para mover datos entre ordenadores. Los datos no van por aquí. Se copia `AlgoData` aparte, o se vuelve a exportar en el ordenador nuevo.
- Para deshacer algo. Esto sube; no borra ni reescribe historia.
- A mitad de un trabajo que está roto. Si `tools/checks.py` sale en rojo, se arregla primero. Un commit roto en GitHub es peor que uno sin subir, porque el siguiente clon empieza desde ahí.

Antes de empezar *(obligatoria)*

Una sola vez en cada ordenador, hay que identificarse contra GitHub. Es lo único que no puede hacer una sesión de Claude por ti: necesita un navegador y un código de un solo uso.

```
gh auth login
```

Responde: `GitHub.com` → `HTTPS` → sí a autenticar git con las credenciales → `Login with a web browser`. Te da un código de ocho caracteres, lo pegas en la página que se abre, y ya está para siempre en ese ordenador.

`gh` está instalado en `~/.local/bin/gh`, no en el sistema. Si `gh: command not found`, añade eso al PATH:

```
export PATH="$HOME/.local/bin:$PATH"
```

Para comprobar que estás dentro:

```
$ gh auth status
github.com
  ✓ Logged in to github.com account sergioguslw (keyring)
```

SQX no tiene nada que ver con esto. Puedes tener la GUI abierta y a media generación: esto no la toca.

Cómo se ejecuta *(obligatoria)*

No es un script de Python, es una **skill**. Se lanza escribiendo esto en Claude Code:

```
/sync
```

No lleva flags. Lo que hace lo decide leyendo el estado del repositorio:

situación	qué hace
no hay repositorio en GitHub	lo crea **privado** y sube todo
hay cosas sin commitear	las agrupa por tema, hace varios commits y los sube
todo está commiteado pero sin subir	solo sube
todo está subido	te lo dice y no hace nada

Tarda segundos. Si prefieres hacerlo a mano, son tres órdenes:

```
git status --porcelain          # qué hay pendiente
python3 tools/depmap.py && python3 tools/checks.py   # tiene que salir verde
git push -u origin --all
```

La diferencia es que la skill además comprueba que no se cuele nada que no debe.

La propuesta de commit de cada mañana: un «sí» y está subido

El commit es tuyo (regla 12): nadie lo hace sin que lo digas. Pero no tienes que agrupar nada a mano. Cada día a las **05:45**, `bin/nightly-commit-proposal.sh` lee lo que hay sin commitear y escribe dos ficheros en `~/Desktop/AlgoData/audit/`:

fichero	para quién	qué lleva
`AAAA-MM-DD-commit.md`	tú	los grupos propuestos (mensaje, nº de ficheros, motivo), la lista **EN CURSO**, la lista **PARADAS** y el resultado de `depmap` y `checks`
`AAAA-MM-DD-commit.json`	el aplicador	lo mismo, en una forma que una orden entiende: por grupo `message`, `body` y `paths`

- **EN CURSO** es lo que no se propone porque otra sesión sigue con ello: ficheros tocados en las últimas 3 horas, o un cambio a medias (un módulo sin su fila de README, una firma cambiada con sus llamadores sin cambiar). Se quedan sin commitear y mañana se vuelven a mirar.
- **PARADAS** es lo que la skill `/sync` nunca sube: `config/machine.yaml`, datos (`.csv`, `.parquet`, `.sqx`, `.db`), secretos, y ficheros de más de 2 MB que no son una captura del manual.
- El agente que escribe la propuesta **no puede** hacer `git add`, `git commit` ni `git push`: está prohibido en su configuración y, además, el script comprueba al acabar que HEAD y el índice no se han movido. Si el árbol está limpio, escribe una sola línea: «Nada que proponer».

Para aplicarla, lees el `.md` y le dices a una sesión de Claude: **«aplica la propuesta de commit»**. Ella ejecuta lo siguiente, que no usa ningún modelo:

```
python3 -m tools.commit_apply --dry-run     # solo enseña lo que haría
python3 -m tools.commit_apply               # hace un commit por grupo y un push
```

opción	qué hace
`AAAA-MM-DD`	aplica la propuesta de otro día (por defecto, la de hoy)
`--dry-run`	no cambia nada; enseña qué haría y qué saltaría
`--only 1,3`	aplica solo esos grupos
`--no-push`	hace los commits pero no sube
`--force`	aplica una propuesta de más de 12 h, o grupos cuyos ficheros cambiaron después; solo si sabes que no ha cambiado nada importante

Antes de cada grupo comprueba que sus ficheros siguen sin commitear y que ninguno se ha modificado **después** de escribirse la propuesta; si alguno sí, **salta ese grupo** y te lo dice. Cada commit lleva solo los ficheros de su grupo (nunca se añade «todo»). Una propuesta de más de 12 horas se rechaza entera. El push usa las credenciales de `gh`; un `git push` a secas falla desde un terminal no interactivo.

Así se ve el resumen (real, un `--dry-run` del 2 de octubre de 2026; la propuesta traía 7 grupos de 51 ficheros y dejaba 116 EN CURSO):

```
DRY-RUN — Propuesta de 2026-10-02
Haría:
  [1] engines: monkey seed is 32 bits and commands take --seed (6 ficheros): haría `git add` + `git commit` de 6 ficheros
  [2] studies: monkey reading and OOS gate follow the nulls seed (7 ficheros): haría ...
  ...
Saltados:
  ninguno
sin push (dry-run o --no-push)
Sigue sin commitear: 180 ficheros (en curso según la propuesta: 116).
```

La última línea es la que importa: lo que **sigue sin commitear** no está en GitHub.

Qué produce *(obligatoria)*

Nada en disco. Cambia dos cosas:

- El **historial de git local**: uno o varios commits nuevos.
- El **repositorio de GitHub**, que pasa a tener lo mismo que tú.

No sobrescribe nada tuyo, no borra ramas y no reescribe commits ya subidos.

Cómo se lee el resultado *(obligatoria)*

Al final te dice qué commits ha hecho. Así salió la primera vez, con todo el atraso acumulado:

```
$ git log --oneline -10
b5680cf Make the analysis half of the project run on Windows
16b0e71 claude: the three analysis skills, and permissions that ask before touching SQX
645a21f knowhow and audit: custom metric columns, export and databank findings
63d85ca docs: manual pages for filters, samples, decay, retests, Monte Carlo, SPP and WFM
628713b strategies: cross-market random-entry study and the trade-level Monte Carlo suite
df75471 tasks: decay, improvement and replication analyses, with the filter and comparison reports
2cabbd7 sqx: exporters for bars, retests, SPP and WFM, and the curation step that applies a verdict
559b37e core: readers for bars, trades, databank stats, SPP profiles and walk-forward matrices
fce8635 OPEN.md: record the layout rename and the retired pipeline maps
```

La línea que importa de verdad es esta:

```
$ git status -sb
## fix/open-issues-2026-09-04...origin/fix/open-issues-2026-09-04
```

Si detrás del nombre de la rama **no** pone `[ahead 3]` ni `[behind 1]`, GitHub tiene exactamente lo que tienes tú. Si pone `[ahead N]`, hay N commits tuyos sin subir. Si pone `[behind N]`, hay N commits en GitHub que tú no tienes, que es lo que pasa cuando has trabajado en el otro ordenador.

Y si el `git status --porcelain` final sale **vacío**, no queda nada sin commitear. Cualquier línea que salga ahí es trabajo que **no** está en GitHub y que **no** aparecerá en el clon.

Un ejemplo completo *(obligatoria)*

Clonar el proyecto en otro ordenador, de principio a fin. En el ordenador de siempre:

```
/sync
```

Te contesta con la URL. En el ordenador nuevo:

```
git clone https://github.com/sergioguslw/AlgoProject.git
cd AlgoProject
cp config/machine.example.yaml config/machine.yaml
```

Ahora **edita** `config/machine.yaml` con las rutas de ese ordenador. Es el único archivo que no viaja en el repositorio, a propósito: es lo único que cambia de una máquina a otra. Si te olvidas, el error te lo dice con todas las letras:

```
/ruta/AlgoProject/config/machine.yaml is missing. Copy config/machine.example.yaml
to it and edit the paths for this machine — it is the only file not in git.
```

Después:

```
python3 -m pip install -r requirements.txt
python3 tools/checks.py
```

Y tiene que salir esto:

```
85 files checked, 0 problems
```

Ahí ya está listo. Los datos no han venido: para analizar, copia `~/Desktop/AlgoData` o vuelve a exportar. El manual en PDF tampoco viene, se reconstruye con `python3 tools/manual.py`.

En **Windows** funciona la mitad de análisis del proyecto y la ventana entera; no la que maneja SQX. Para tener la ventana en un PC con Windows:

1. Python 3.10–3.13 de python.org, marcando *Add python.exe to PATH*.
2. `git clone` del repositorio y copia la carpeta `AlgoData` entera donde quieras.
3. En la carpeta del proyecto, `copy config\machine.example.yaml config\machine.yaml` y cambia tres líneas, con barras normales:

```
data_root: C:/Users/<tu-usuario>/Desktop/AlgoData
sqx_master: C:/none      # tiene que estar, nunca se abre en Windows
sqx_worker: C:/none
```

y borra el bloque `sqx_workers:` y el de `strategy_pools:` enteros. 4. `python -m pip install -r requirements.txt`. 5. Doble clic en `bin\algoui.cmd`.

`machine.yaml` no viaja en git a propósito: es el único fichero que cambia de una máquina a otra.

Qué NO te dice *(obligatoria)*

- **No te dice si el código está bien.** Solo que pasa las comprobaciones mecánicas: longitudes, docstrings, rutas absolutas, READMEs. Eso es `/audit`, y ni eso juzga si un análisis concluye bien.
- **La propuesta no es un juicio de calidad.** Agrupa por tema y deja fuera lo que parece a medias, pero lo hace mirando fechas y ficheros, no ejecutando cada cosa: lee el `.md` antes de decir que sí.
- **No decide qué rama es la buena.** Sube las que haya y las deja ahí. Fusionar `master` con una rama de trabajo lo decides tú.
- **Un repositorio subido no es una copia de seguridad de tu trabajo con SQX.** Los proyectos, las databanks y las estrategias no están aquí. Si se pierde la instalación de SQX, GitHub no te la devuelve.
- **No garantiza que el clon dé los mismos números.** Da el mismo código. Los mismos resultados necesitan además los mismos datos, y los datos van por su cuenta.

Si algo falla

`gh: command not found` — `gh` está en `~/.local/bin`. `export PATH="$HOME/.local/bin:$PATH"`.

`You are not logged into any GitHub hosts` — falta `gh auth login`. Tienes que hacerlo tú, en tu terminal, porque hace falta el navegador.

`Updates were rejected because the remote contains work that you do not have` — has trabajado en el otro ordenador y ahí hay commits que aquí no. `git pull --rebase` y vuelve a subir. **Nunca** **`--force`**: eso borra de GitHub el trabajo del otro ordenador.

`checks.py` en rojo — se arregla antes de subir. No hay prisa que justifique subir algo roto.

114. Revisión estructural semanal — dónde se está acumulando deuda en el código

Qué pregunta responde

Cada semana: **¿el código necesita una refactorización, dónde, y está mejor o peor que la semana pasada?** Muchos agentes cambian el proyecto a la vez, y la deuda no aparece de golpe: aparece como dos copias de la misma fórmula que poco a poco dejan de coincidir, una capa que empieza a importar de la de arriba, o código que ya nadie llama. El auditor diario te dice lo que está roto hoy; esta revisión te dice hacia dónde deriva la estructura.

Mira siete cosas: **duplicación** (sobre todo métricas como Sharpe o drawdown calculadas de dos maneras), **código muerto**, **corrección cuant** (fugas IS/OOS, unidades, semillas, zonas horarias), **convenciones**, **robustez** (ficheros compartidos sin cerrojo, esperas sin límite), **arquitectura** (capas y dependencias) y **experiencia de desarrollo** (tests, checks, peso del contexto de los agentes).

Cuándo lo usas, y cuándo no

- **Sí:** el sábado por la mañana, para decidir qué encargo de refactorización va antes.
- **Sí, a mano:** cuando te preguntes si el código «está para una limpieza».
- **No para saber si algo está roto ahora:** eso es el auditor diario (`AlgoData/audit/<fecha>.md`).
- **No toca nada.** Solo escribe su informe: no cambia código, no hace commits, no corre los tests, no toca SQX, MT5 ni gorilla. Lo que encuentra lo decides tú.

Antes de empezar

Nada. Corre solo desde cron el **sábado a las 04:30**, cuando ya ha terminado la cadena nocturna (acaba a las 04:15). No necesita SQX ni ningún worker.

Cómo se ejecuta

Solo, desde cron. A mano, lo mismo:

```
bin/weekly-refactor-scout.sh
```

flag	obligatorio	qué hace
`--dry-run`	no	enseña las rutas y el binario de Claude que usaría, y no corre nada

Dos mitades. La **mecánica** (vulture, ruff, `tools/checks.py`, cuántas excepciones de capas quedan, líneas por paquete y ficheros más tocados en la semana) tarda **1 min 25 s** y no usa ningún modelo. La **del agente** (`refactorScout`, Sonnet) reparte las siete dimensiones entre siete subagentes Sonnet y junta lo que vuelven; tiene un tope de 3 horas. Lo que gasta queda en `AlgoData/logs/agent-spend.csv`, y si falla te llega una notificación y un `ERROR-*.txt` en `AlgoData/audit/`.

Qué produce

Todo en `AlgoData/audit/refactorScout/`, nada en el repositorio:

- `<fecha>-mechanical.md` — las cifras de la mitad mecánica.
- `<fecha>.md` — **el informe que lees**, en español.
- `findings.json` — la lista de hallazgos con un identificador estable; se sobrescribe cada semana y es lo que permite decir «nuevo», «sigue», «empeora» o «arreglado».

El log está en `AlgoData/logs/weekly-refactor-scout.log`.

Cómo se lee el resultado

El informe tiene cinco partes:

1. **Veredicto** — tres líneas: si hace falta refactorizar ya, dónde, y cuántos hallazgos son nuevos, siguen, empeoran o se arreglaron.
2. **Top 5** — las cinco refactorizaciones que más rinden, cada una con su coste (S pequeño, M medio, L grande, y cuántos ficheros toca) al lado de lo que gana, y si cambia números con los que decides (un Sharpe, un drawdown, un filtro).
3. **Por dimensión** — una tabla por cada una de las siete: estado, hallazgo, `fichero:línea`, gravedad y esfuerzo.
4. **Arreglado esta semana.**
5. **No comprobado** — lo que no pudo mirar y por qué.

La **gravedad** tiene tres valores: *bug ahora* (algo da un número o un resultado equivocado hoy), *latente* (todavía no falla, pero fallará cuando cambie algo) y *fricción* (no falla, pero hace más lento o más caro cambiar el código). Lo que vale la pena mirar primero es lo que *empeora* y lo que es *bug ahora*. Un hallazgo que *sigue* semana tras semana es un candidato a encargo.

Un ejemplo completo

La primera corrida de la mitad mecánica, el 2026-10-16:

```
$ bin/weekly-refactor-scout.sh --dry-run
root   /home/sergioguslw/Desktop/AlgoProject
log    /home/sergioguslw/Desktop/AlgoData/logs/weekly-refactor-scout.log
out    /home/sergioguslw/Desktop/AlgoData/audit/refactorScout
claude /home/sergioguslw/.local/bin/claude
```

De `2026-10-16-mechanical.md`:

```
## vulture (confianza 60)
total: 297
core/assetwrite/__init__.py:164: unused function 'rename_cost' (60% confidence)
core/significance.py:163: unused function 'min_track_record' (60% confidence)
...
## Excepciones de capas (tools/layering.py ALLOW)
19
## tools/checks.py
## ratchet: 1
  PLR0913: 231 -> 232 (ratchet: may not rise; see CODESTYLE.md §12)
1687 files checked, 3 problems
```

Se lee así: vulture marca 297 candidatos a código muerto con confianza 60, y el agente comprueba uno por uno si alguien los llama antes de darlo por muerto. Quedan 19 excepciones a las reglas de capas, y la meta es 0. `checks.py` falla en 3 cosas. La línea `PLR0913` dice que hay una función más con demasiados parámetros que la semana anterior.

Qué NO te dice

- **No te dice si una estrategia es buena**, ni nada de la configuración de SQX: los proyectos y sus tareas son tuyos y nunca son un hallazgo.

- **No ejecuta los tests.** Comprueba qué cubren, pero no si pasan hoy.
- **Un □ es una sospecha, no un hecho.** Solo □ está comprobado leyendo el código.
- **No arregla nada.** Pasar un hallazgo a encargo es decisión tuya (`/encargo`).